VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



LABORATORY 2 REPORT

Project

Spa & Beauty Center Website

Instructor: Nguyễn Đức Thái
Student: Nguyễn Minh Quân – 2353003
Class: CC01

HO CHI MINH CITY, OCT 2026

Table of Contents

1 Project Overview **1**

2 Database Overview **1**

3 Implemented Features **3**

4 Session and Cookie Usage **4**

5 Testing and Results **5**

6 Problems Encountered and Resolutions **5**

A Supporting Screenshots **6**

1 Project Overview

Extending the frontend developed in Laboratory 1 by connecting the service catalogue to a database and introducing user accounts and access control. Visitors can browse and search spa treatments, registered customers can access a personal dashboard, and administrators can manage the services displayed on the website.

The application runs locally in XAMPP and uses Apache, PHP 8.2.12, MariaDB 10.4.32, HTML, CSS, and JavaScript. Its homepage is `public/index.php`, located within the project directory under the Apache web root. The work completed in this lab provides the database and account features on which the subsequent laboratory work will build.

2 Database Overview

The application stores its data in the `spa_beauty_center` database. The tables use the InnoDB storage engine and `utf8mb4` character encoding. The database schema and initial records are included in `database/spa_beauty_center.sql`.

Table	Stored information	Initial records
`users`	Customer and administrator accounts, including name, unique email address, password hash, and role.	No initial accounts
`categories`	Service category names, unique slugs, and descriptions.	4 categories
`services`	Category, service name, unique slug, descriptions, price, duration, image filename, and featured status.	6 services

Each table has an auto-incrementing primary key. The relationship between categories and services is one-to-many: a category may contain several services, while each service belongs to one category. The foreign key `services.category_id` references `categories.id`. A category cannot be deleted while services still reference it. User accounts are stored separately from this relationship.

Service prices are stored as positive whole amounts in Vietnamese dong (VND). Duration is represented by minimum and maximum values in minutes, allowing the website to display either a fixed duration or a range. The six initial services are Swedish Massage, Deep Tissue Massage, Radiance Facial, Hot Stone Therapy, Manicure & Pedicure, and Aromatherapy. Their image fields refer to photographs in `public/images/`.

Accounts are created through customer registration or the local administrator setup tool. The database does not include a default administrator account or password. Figures 1–3 show the service records, categories, and account structure in phpMyAdmin.

Database Screenshots

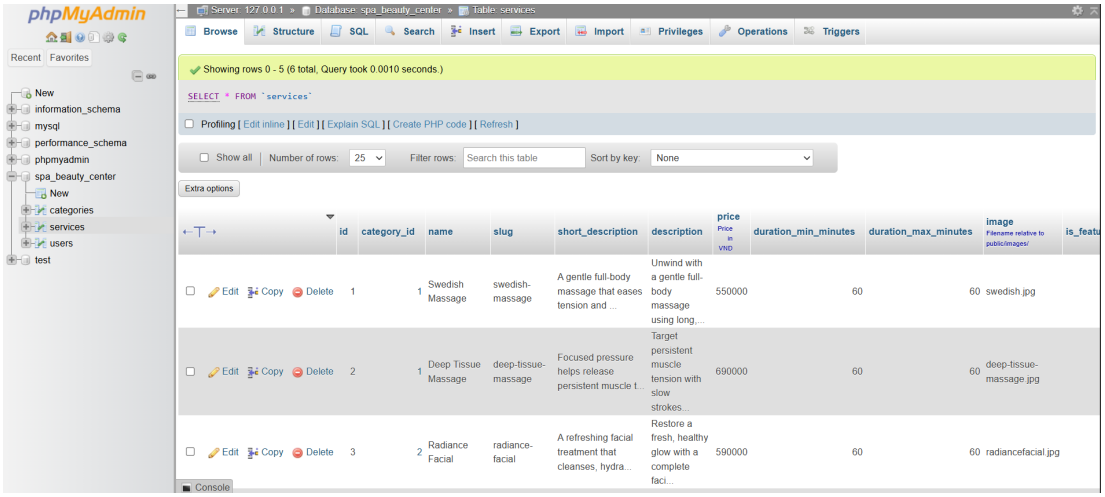


Figure 1: Service records, including prices, durations, and image filenames.

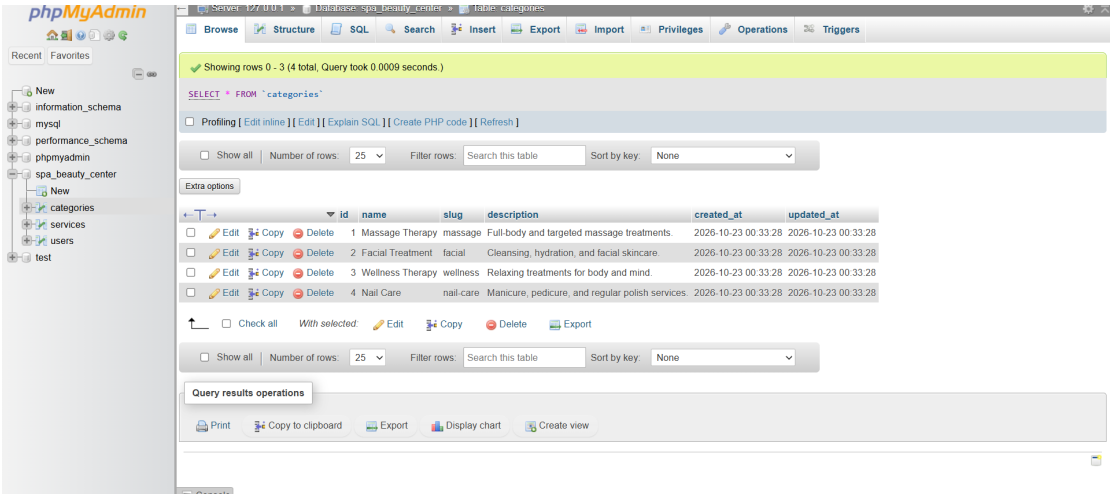


Figure 2: The four categories used to organise spa services.

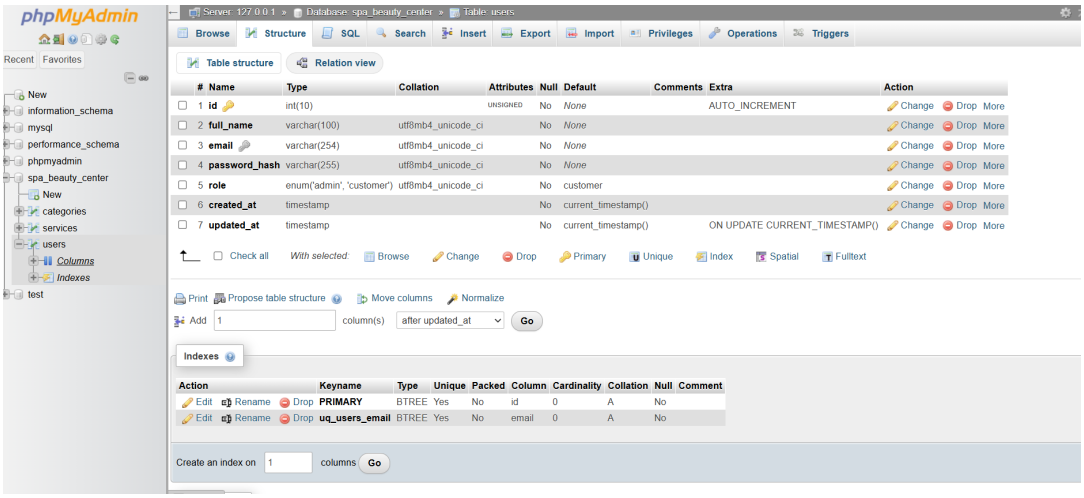


Figure 3: Users table structure, showing account fields, the password-hash field, roles, and indexes.

3 Implemented Features

3.1 Database Integration (Tasks 1–2)

The application connects to MariaDB through PHP Data Objects (PDO), with shared connection settings used across the website. The service catalogue, category filters, featured treatment, and service details pages retrieve their content from the database. This allows changes to service records to appear on the public pages without editing the page markup. If the database is unavailable, the application displays an error message without exposing connection details or SQL exceptions. Figures 5 and 6 compare a stored service record with its corresponding details page.

3.2 Service Management (Task 3)

The administrator area supports the create, read, update, and delete (CRUD) operations for services. Administrators can view the service list and individual records, add treatments, edit existing information, and remove services. Server-side validation checks required fields, category selections, prices, durations, slugs, and permitted image filenames. Queries containing user input use prepared statements. Successful changes are followed by a redirect and a confirmation message. Deletion requires confirmation and a POST request, and cross-site request forgery (CSRF) protection is applied to service changes.

3.3 Registration and Authentication (Task 4)

Customer registration checks the submitted name and email address, password requirements, password confirmation, and whether the email address is already registered. Passwords are stored as hashes using `password_hash()` and checked during login using `password_verify()`. After successful login, customers can access a protected account dashboard. Logout uses a CSRF-protected request and invalidates the authenticated session.

3.4 Sessions and Display Preferences (Task 5)

PHP sessions keep customers authenticated as they move between pages. A separate cookie stores the selected light or dark appearance. This preference persists after logout and does not contain account information. Session and cookie behaviour is described in more detail in Section 4.

3.5 Role-Based Access Control (Task 6)

Access to service management is restricted to administrators. The server checks the account's current database role whenever a management page or action is requested. Customers and unauthenticated visitors are therefore denied access even if they enter a management URL directly or submit a request outside the normal interface. For this local laboratory deployment, the management area also accepts requests only from the computer hosting the application.

4 Session and Cookie Usage

The application uses two cookies with distinct purposes. Authentication data is held in a server-side session, while the appearance preference is stored separately in the browser.

Cookie	Purpose	Lifetime
`spa_session`	Identifies the server-side session containing the authenticated user ID, CSRF tokens, and feedback messages.	Browser-session cookie. Logout destroys the server-side session and expires the cookie.
`spa_theme`	Stores either `dark` or `light` to retain the chosen appearance.	30 days after selection or migration; retained after logout.

The session cookie uses `HttpOnly`, which prevents JavaScript from reading it. The theme cookie remains accessible to JavaScript so that the appearance toggle can update the preference. Both cookies use `SameSite=Lax`; the `Secure` attribute is enabled when the application is served over HTTPS. The local test environment uses HTTP.

Successful login regenerates the session identifier. Protected requests use the stored user ID to retrieve the account's current role from the database, so a role change takes effect on the next request. The theme cookie contains no passwords, password hashes, or role information and has no effect on access permissions.

The implementation does not enforce an application-level idle timeout. A browser-session cookie also does not guarantee logout when a browser restores previously opened tabs. Explicit logout provides the means of invalidating the authenticated session. Figures 13–17 show the preference cookie before and after reload, navigation, and logout.

5 Testing and Results

Integration testing for Task 7 used automated tests that checked database access, service management, authentication, sessions, display preferences, and access restrictions. Node.js 24.13.1 was used for JavaScript testing, and a Chrome workflow checked the appearance controls and cookie behaviour in the browser.

Test area	Coverage	Result
Database integration	Connection, catalogue and details pages, malformed links, and database-outage responses.	128 checks passed
Service management	CRUD operations, changes reflected on public pages, validation, output escaping, CSRF protection, and deletion confirmation.	183 checks passed
Authentication and sessions	Registration, unique email addresses, password hashes, login, session persistence and ID regeneration, logout, and rejection of invalidated sessions.	188 checks passed
Access control	Rejection of unauthenticated and customer requests, manipulated role values, administrator actions, role changes, and deleted accounts.	310 checks passed
Theme preference	Cookie values, persistence, expiry, migration, preference priority, and behaviour when browser storage is blocked.	8 tests passed
Browser workflow	Theme toggle, cookie expiry and attributes, page reloads, navigation, initial theme rendering, and invalid-cookie handling.	Passed

The final test run recorded **809 PHP checks and 8 JavaScript tests passed**, together with a successful Chrome workflow. Syntax checks also passed for 27 PHP files and three JavaScript files used by the application and browser tests. Registration, login, logout, and service management were tested through HTTP requests to the PHP forms. The Chrome workflow focused on visual behaviour and cookies; it did not exercise every form through the browser interface.

The test suites were run sequentially through `tests/run-lab2.ps1`. Temporary accounts and services were removed after testing. Before-and-after comparisons found no changes to the original account and service records, and the database contained zero users, four categories, and six services both before and after the run. These checks helped confirm that the tests did not alter the initial application data.

6 Problems Encountered and Resolutions

A review identified a potential timing issue in browser tests that repeatedly navigated to the same URL. The test could inspect the previous page before the new document had finished loading, producing results based on outdated page content. The test was revised to wait for the new document's load event before checking the page. The combined test suite was then run again.

A further concern was that the separate test suites used the same development database. Simultaneous execution could cause temporary test records to interfere with one another and with the checks on the original data. The combined runner addresses this by executing the suites in sequence and returning a failure status if a suite fails.

No failures were reported in the final integrated run. The results cover the laboratory workflows listed in the testing section; deployment beyond the local XAMPP environment was outside the scope of this lab. The database, authentication, and service management features provide a basis for the next stage of the semester project.

A Supporting Screenshots

The following screenshots illustrate the database structure and the main application workflows described in this report.

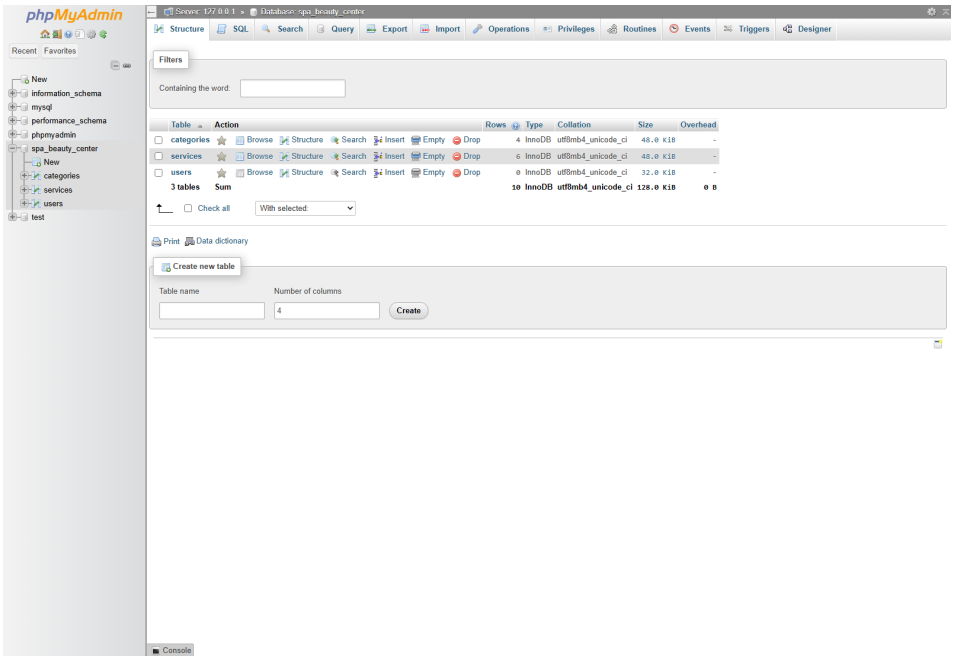


Figure 4: Database overview in phpMyAdmin, showing the three application tables.

Task 2: Database Content and Website Display

Figures 5 and 6 show the Swedish Massage record in the database and the information displayed on its details page.

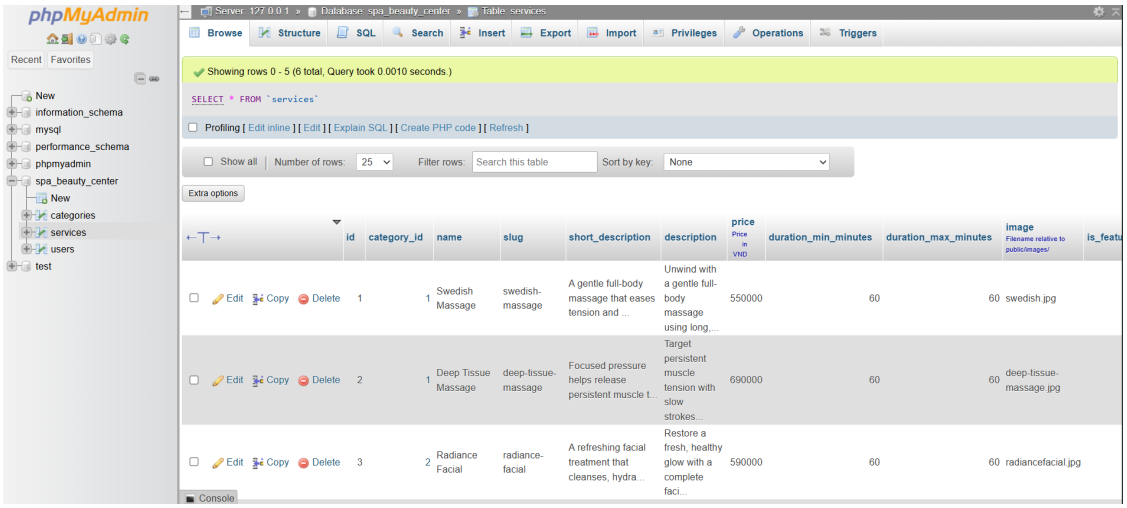


Figure 5: The Swedish Massage record and its stored values in MariaDB.

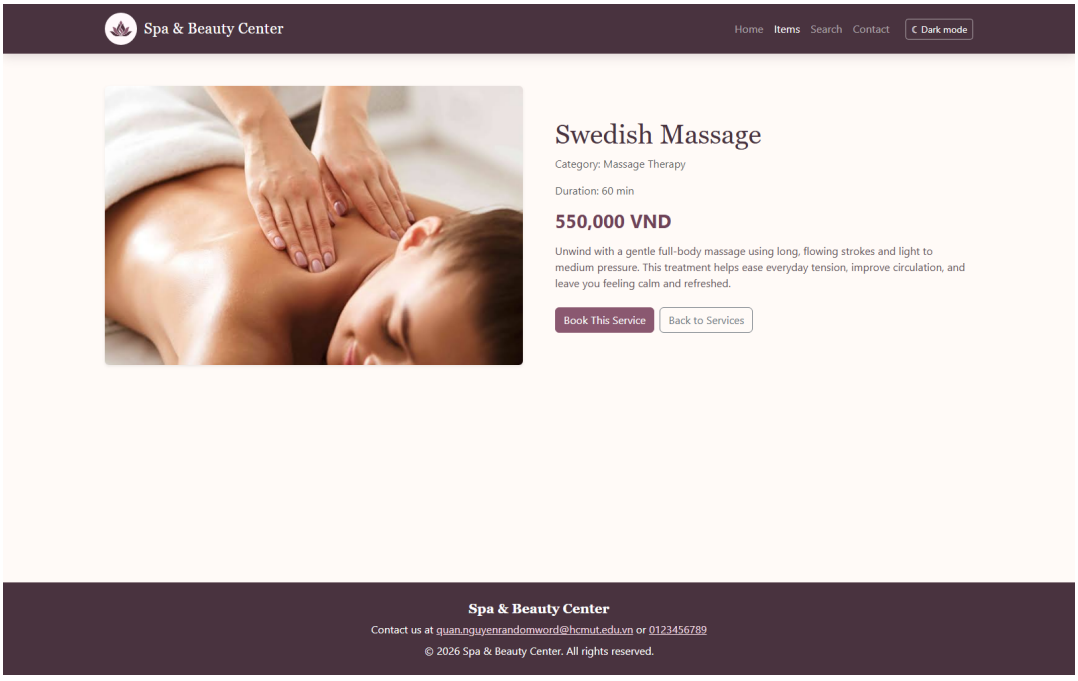


Figure 6: The corresponding Swedish Massage details page.

Field	Stored value	Website display
Service name	Swedish Massage	Swedish Massage
Price	550000	550,000 VND
Duration	Minimum 60, maximum 60 minutes	60 min

The displayed name, price, and duration match the database record. The website formats the price with a thousands separator and a currency label while preserving the stored amount.

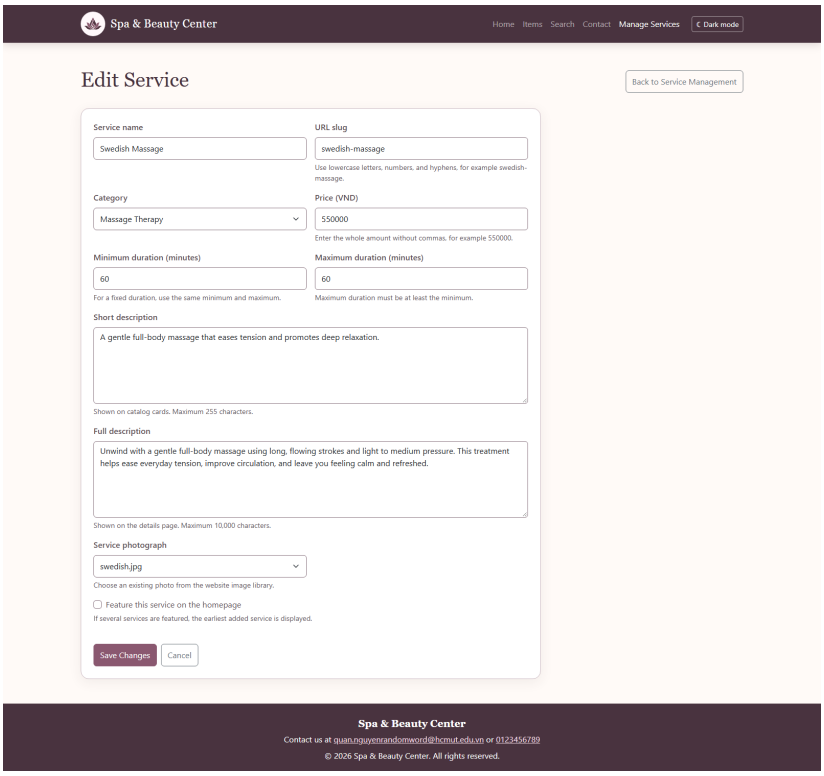


Figure 7: Administrator form for adding or editing a spa service.

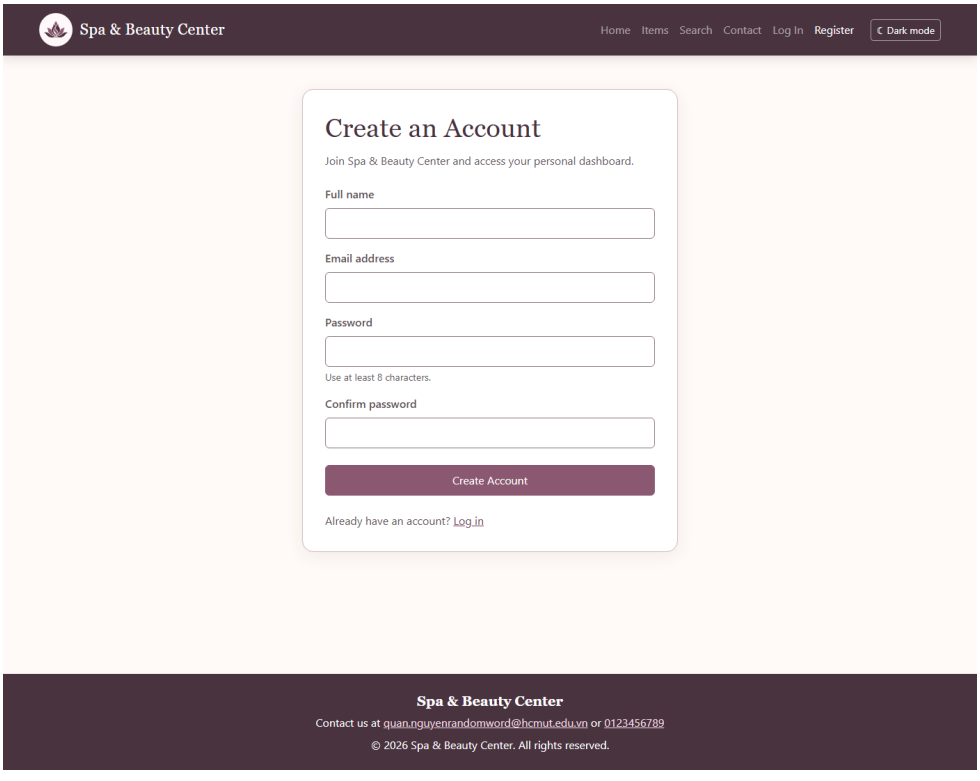


Figure 8: Customer registration form with account details and password confirmation.

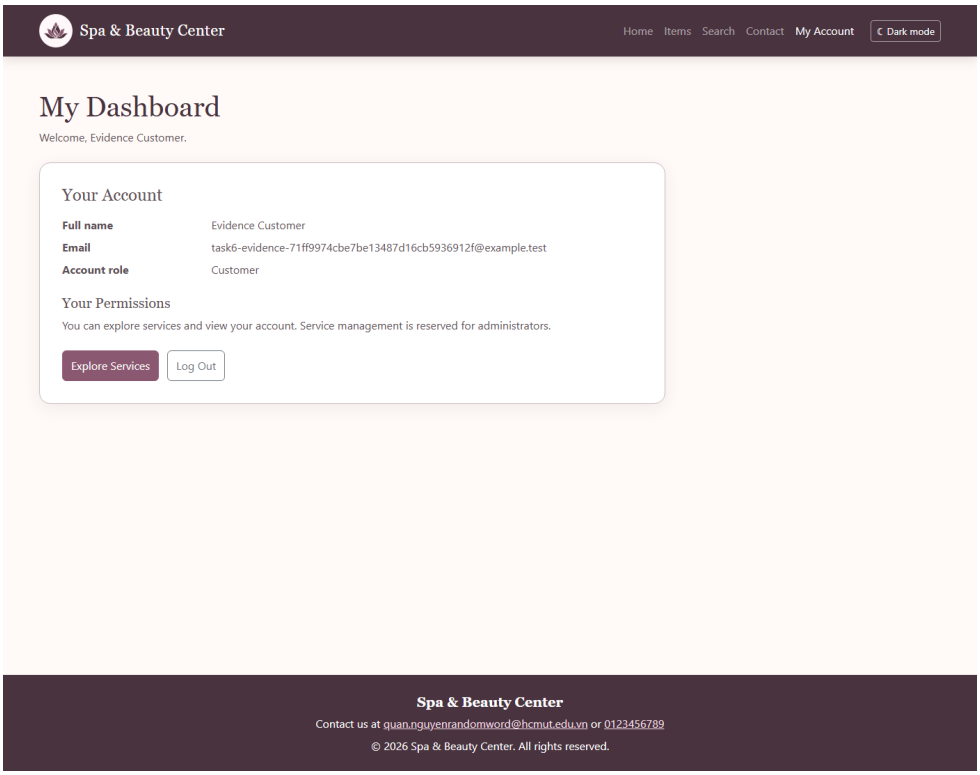


Figure 9: Customer dashboard after successful authentication.

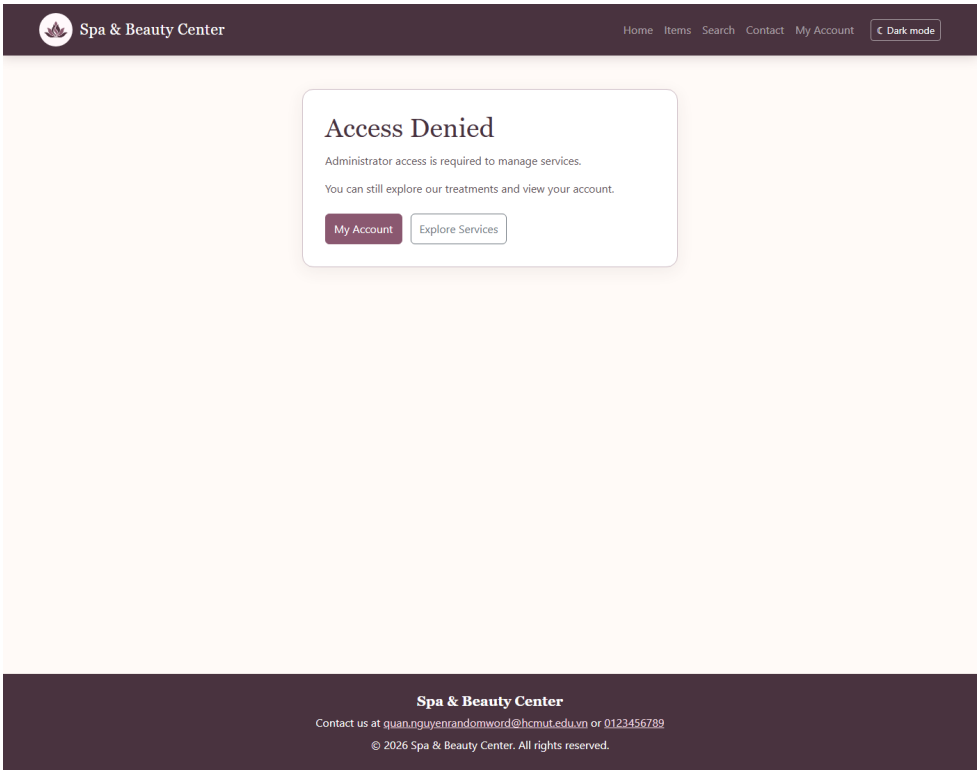


Figure 10: Access denied when a customer requests a service management page.

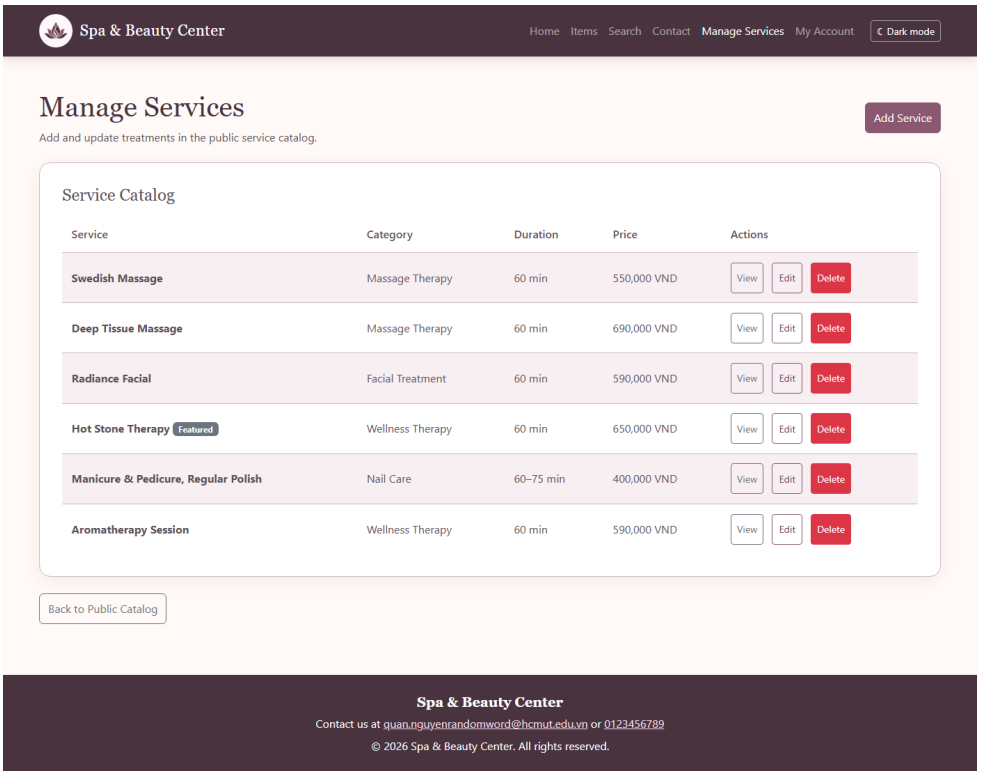


Figure 11: Administrator service list with controls to add, view, edit, and delete records.

Task 5: Cookie Storage

The evidence was captured in an isolated Chrome profile. Each pair shows separate website and native DevTools views of the same browser state; only `spa_theme` is shown in the cookie table. Cookie views focus on the Application tab, filter, and preference row; original full-resolution screenshots are retained.

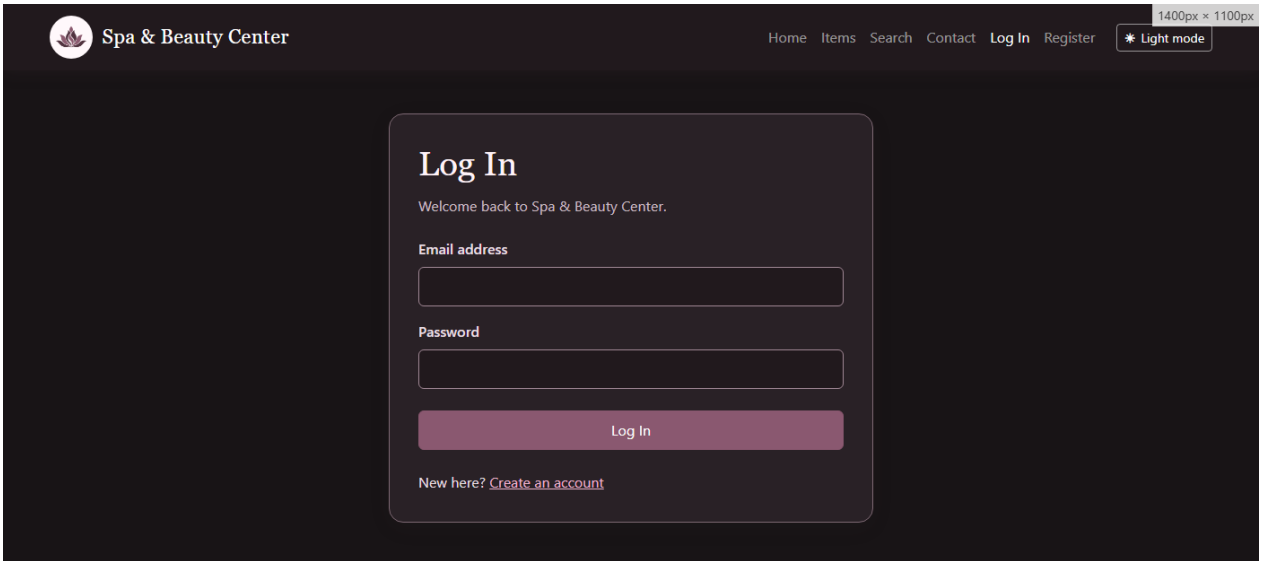


Figure 12: Step 1 – Selecting dark mode writes the appearance preference.

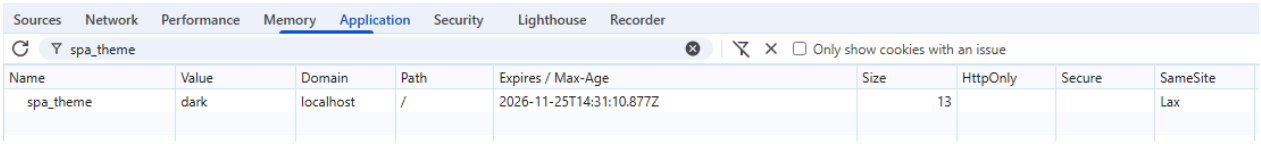


Figure 13: Native Application/Cookies view: the saved preference and expiry.

Attribute	Observed value
Name / value	`spa_theme` / `dark`
Domain / path	`localhost` / `/`
Expiry (UTC)	25 November 2026, 14:31:10.877
Configured lifetime	2,592,000 seconds (30 days) from selection
SameSite	`Lax`
HttpOnly / Secure	Both false: a JavaScript-readable preference on local HTTP

The expiry is configured for 30 days; this capture does not claim observation over 30 elapsed days. The cookie contains no account identity or credentials.

Task 5: Reload and Navigation

The collector reloaded the page, confirmed that dark mode was restored, then navigated to the Swedish Massage details page in the same browser profile.

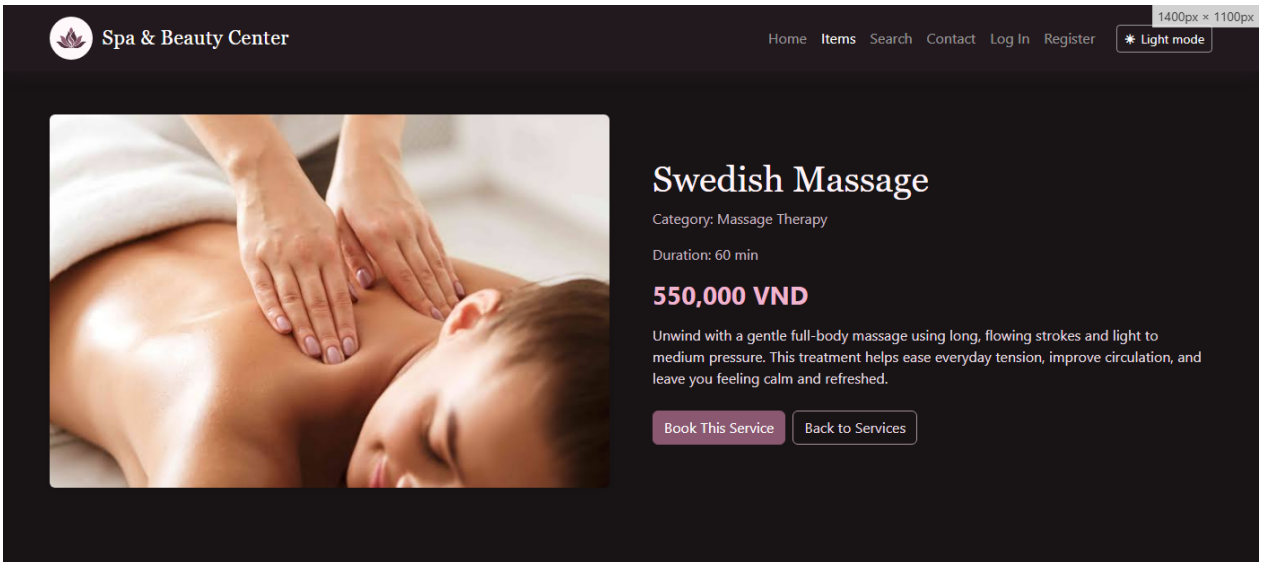


Figure 14: Step 2 – Dark mode remains active on the service details page.

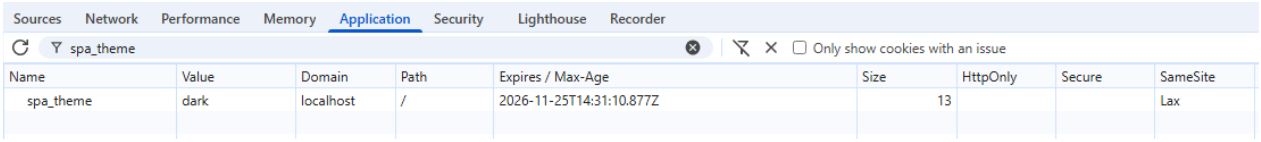


Figure 15: After reload/navigation, the preference and original expiry are unchanged.

Check	Before	After reload/navigation
Preference	`dark`	`dark`
Expiry (UTC)	25 Nov 2026, 14:31:10.877	25 Nov 2026, 14:31:10.877
Seconds remaining	2,592,000	2,591,997

The remaining lifetime decreases as time passes. Reloading or opening another page does not reset the expiry. The collector compared the exact cookie value and expiry returned by Chrome, not just the visible page colors.

Task 5: Preference After Logout

A temporary customer registered and logged in through the normal PHP forms. Login rotated the session identifier. After the CSRF-protected logout, requesting the protected dashboard redirected to the login page shown below.

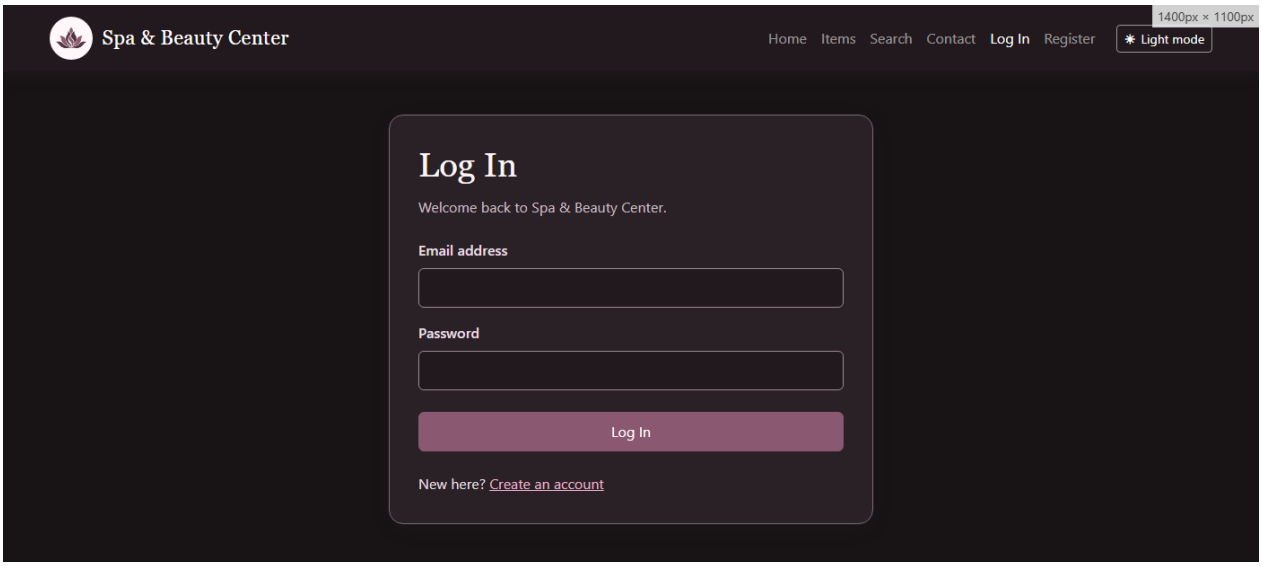


Figure 16: Step 3 – Authentication has ended, but the login page stays dark.

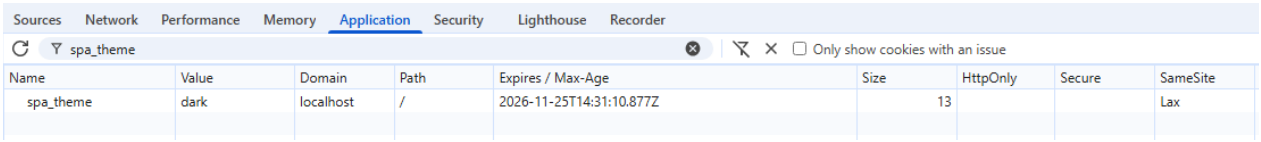


Figure 17: The independent preference cookie survives logout with its original expiry.

The cookie remains `spa_theme=dark`, expiring on 25 November 2026 at 14:31:10.877 UTC (2,591,992 seconds remaining at the final check). The authentication session ends independently of this cosmetic preference. The login page may create a fresh anonymous session for CSRF protection; logout does not require the browser to contain no session cookie at all.

Session identifiers, passwords, and CSRF values are excluded from the captures. The temporary account and isolated browser profile were removed after the successful capture. Safe cookie attributes are recorded in `docs/screenshots/task5-cookie-metadata.json`.